IE437 · DATA-DRIVEN DECISION MAKING AND CONTROL — LECTURE 5

Data-Driven Design Optimization — Surrogate-Based

A surrogate you optimise against will be exploited where it is wrong

Jinkyoo Park · KAIST

Offline, part 1 of 2 — approximate the function, then search it

HANDOFF

The handoff — optimisation with no oracle

Lecture 4 could *query* the expensive function whenever it wished. Take the query away and the same loop turns on itself.

Where we are — the query is taken away

STAGES	MODEL	AGENTS
static ● ○	data-driven ○ ●	single agent ● ○

	MODEL-BASED (CERTAIN)	DATA-DRIVEN (UNCERTAIN)
Static, single	optimisation *(Lec 1)*	belief *(Lec 2–3)* · acting on belief *(Lec 4)* · **design from a fixed dataset *(Lec 5–6)***

Lecture 4 built a posterior over an unknown f and then *acted* — chose a point, queried it, folded the answer back in. Often you cannot. The data is already collected and **fixed**: a database of proteins and their activity, of alloys and their strength, of accelerator layouts and their latency. No new experiments; the budget was spent, or the wet lab is closed, or one evaluation costs a month.

Lecture 4 leaves us $\arg\max_x f(x)$ with a GP — and this lecture **removes the oracle**. Same cell of the cube: static, data-driven, single agent, and the same goal as Lecture 1. What has been taken away is not the model but the right to check.

Four ways to optimise a black box, and what every one of them needs

what the source lecture spends fifteen slides establishing

METHOD	WHAT IT DOES	WHAT IT COSTS
Gradient ascent on a proxy	fit f_θ to the data so far, step $x_{t+1} = x_t + \eta \nabla_x f_\theta(x_t)$, evaluate	one query per step
Genetic algorithms	population, truncation selection of the top E, crossover, mutation	N queries per generation
CMA-ES	sample $x_i \sim \mathcal{N}(\mathbf{m}_t, \sigma_t^2 I)$, rank, move the mean and shrink σ	n queries per generation
Bayesian optimisation *(Lec 4)*	GP posterior, then $x_{t+1} = \arg\max_x A_t(x)$	one query per round
Policy gradient	learn $\pi_\theta(x)$ by $\nabla_\theta \mathbb{E}_{x\sim\pi_\theta}[f(x)] = \mathbb{E}[\nabla_\theta \log \pi_\theta(x)\, f(x)]$	n queries per round

Every one of the five is a loop, and every loop closes through an evaluation of the real f.

The problem, and the two routes out of it

The offline problem is that loop with one line struck out — and it is exactly Lecture 1's goal under a harsh new constraint:

$$\text{find} \quad x^* = \arg\max_x f(x) \qquad \text{with only a fixed dataset } D = \{(x_1, f(x_1)), \ldots, (x_N, f(x_N))\}$$

This is **offline model-based optimisation**. Two routes exist, and they open the next two lectures:

1 · SURROGATE-BASED — *THIS LECTURE*

Approximate f from D, then optimise **over the surrogate**. A *forward* model, and a *search*.

2 · GENERATIVE-BASED — *LECTURE 6*

Learn the inverse map $p(x \mid y)$ and **sample** designs that are already good. An *inverse* model, and a *draw*.

The source deck's figure is a picture of one deleted arrow: real data flows into an offline optimiser, a design flows out to superconductors, DNA sequences, proteins and robot morphologies — and the return arrow, from the design back to the world, is struck through: **no additional interactions**.

The thesis — the surrogate's blind spots are where you will be sent

An optimiser turned loose on a learned surrogate will seek out exactly the inputs where the surrogate is wrongly optimistic.

It sounds like Lecture 4 again — fit a model of f, optimise it. Offline the danger is far sharper. In BO a promising point could be *checked* by querying it, and a wrong belief was corrected within one round. Here a design the surrogate loves but that is in fact worthless is returned as the answer.

So the whole lecture is one failure and its cure: the surrogate **overestimates** where it has no data, the optimiser **exploits** that overestimation, and the fix is to make the surrogate deliberately **conservative** exactly where it will be attacked. Keep that sentence; Lecture 12 will need it, one axis over.

The roadmap — four questions

Q1 What changes offline? → Q2 Why does it fail? → Q3 How do we fix it? → Q4 What else helps?

- **Q1 — What changes when optimisation goes offline?** No oracle, so no way to check a candidate before returning it.
- **Q2 — Why does the naive "fit and ascend" fail?** Overestimation off the data, on a narrow manifold of valid inputs.
- **Q3 — How do we fix it?** Conservative objective models — penalise the surrogate where the optimiser attacks.
- **Q4 — What else helps?** Honest uncertainty (NEMO) and local smoothness (RoMA).

ACT 1

The naive approach

Q1. Supervised learning, then optimisation. Two lines, and on paper it should work.

The obvious method — fit a proxy, then climb it

Q1 What changes offline? → Q2 Why does it fail? → Q3 How do we fix it? → Q4 What else helps?

$$\textbf{Step 1.}\quad \theta^* = \arg\min_\theta \frac{1}{N}\sum_{i=1}^{N}\left(f_\theta(x_i) - f(x_i)\right)^2 \qquad \textbf{Step 2.}\quad x^* = \arg\max_x f_{\theta^*}(x)$$

Fit a neural network f_θ to D — this is the **surrogate**, or proxy — and then, because f_θ is differentiable where the real f was not, run gradient ascent on it.

ONLINE *(LECTURE 4)*

```
for t = 1 … T−1:
  train f_θ on D_t
  x_{t+1} = x_t + η ∇_x f_θ(x_t)
  y_{t+1} = f(x_{t+1})          ← query
  D_{t+1} = D_t ∪ {(x_{t+1}, y_{t+1})}
```

OFFLINE *(THIS LECTURE)*

```
train f_θ on D                  ← once
for t = 1 … T−1:
  x_{t+1} = x_t + η ∇_x f_θ(x_t)

return x_T
```

One line deleted, and with it every correction. The surrogate is fitted once and never contradicted again.

What "high-dimensional" means here

the six Design-Bench tasks these methods are measured on

TASK	DIMENSION	TYPE	DATASET SIZE
Superconductor	81	continuous	21,263
GFP *(green fluorescent protein)*	238	categorical (20)	5,000
MoleculeActivity	1,024	binary	4,216
HopperController	5,126	continuous	3,200
AntMorphology	60	continuous	25,009
DKittyMorphology	56	continuous	25,009

Five thousand designs in a 5,126-dimensional space. Whatever the surrogate believes about that space, **almost all of it was never checked against anything** — and gradient ascent is free to walk in any of those directions. *(Trabucco et al., Design-Bench, 2022)*

ACT 2

Why it fails

Q2. Two facts collide, and the optimiser is standing exactly where they meet.

Two problems, not one

Q1 What changes offline? → **Q2 Why does it fail?** → Q3 How do we fix it? → Q4 What else helps?

PROBLEM 1 — EXTRAPOLATION

The trained model is only valid **near the training distribution**, so its error off that distribution is large and unbounded.

And yet the whole point of the exercise is to return a design *better than anything in* $\mathcal{D}$ — so **we must extrapolate**. The failure is not incidental to the task; it *is* the task.

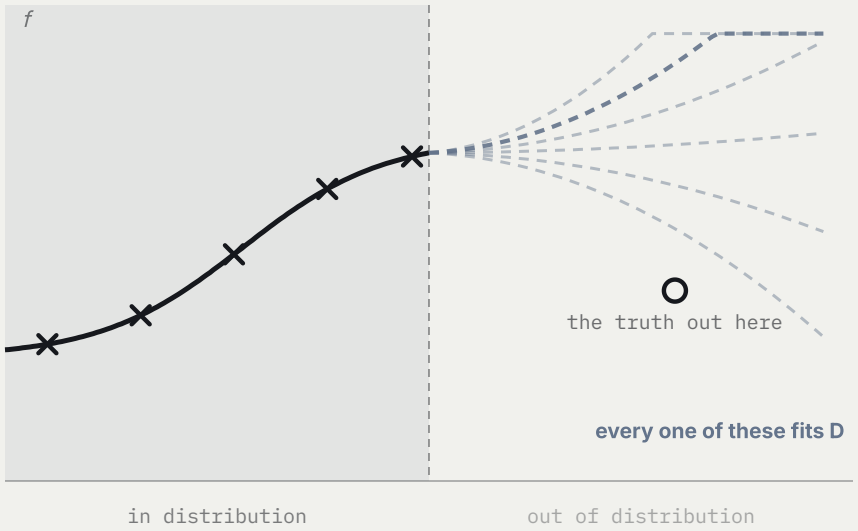
PROBLEM 2 — THE VALID MANIFOLD

Searching for the input that maximises the proxy is easy: gradient ascent. But only **a thin sliver of the input space is valid** at all — real molecules, foldable proteins, buildable layouts.

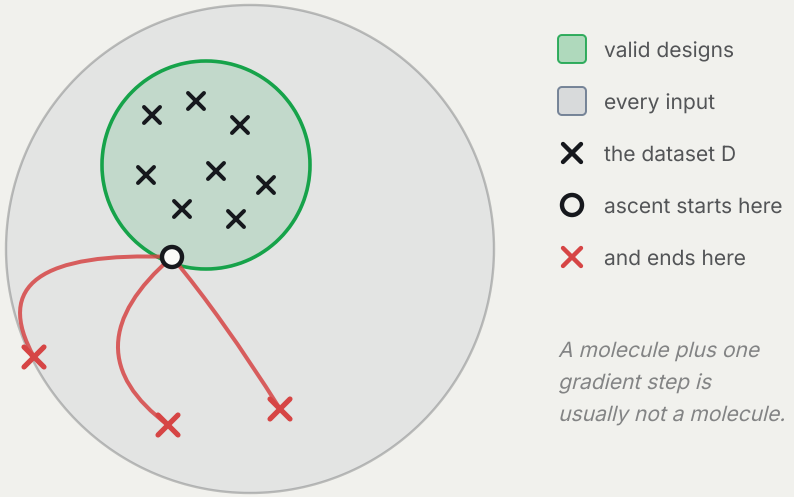
In high dimensions, ascent steps almost surely leave that sliver, and a design off it is **not merely poor but meaningless**.

The two want different cures. Problem 1 is about the *values* the surrogate reports; Problem 2 is about the *set* the optimiser is allowed to move in. Act 3 attacks the first. Lecture 6 — which searches inside a learned generative model — attacks the second.

The two pictures



PROBLEM 1 — D DOES NOT DETERMINE f OFF D



PROBLEM 2 — AND ASCENT LEAVES THE VALID SET

Left, the dataset does not determine f off the data: **every one of those dashed continuations fits D equally well**, and the fitted surrogate is whichever one the architecture happens to prefer. Right, the valid inputs are a small disc inside a large space; ascent starting inside it leaves almost immediately, and the returned designs are not molecules at all.

The optimiser is an adversary

Gradient ascent on f_θ does not merely stumble into the bad region. It **searches for it**, because the bad region is where f_θ is highest.

THE MECHANISM HAS A NAME — Goodfellow, Shlens & Szegedy, 2014

A photograph classified "panda" at 57.7 % confidence, plus 0.007 times a crafted noise field, is classified **"gibbon" at 99.3 % confidence**. Nothing about the image changed that a person could see; the perturbation was simply chosen by ascending the model's own gradient.

Gradient ascent on a learned f_θ is **the identical mechanism**, pointed at a regression head instead of a classifier: it manufactures inputs the model rates highly and the world does not.

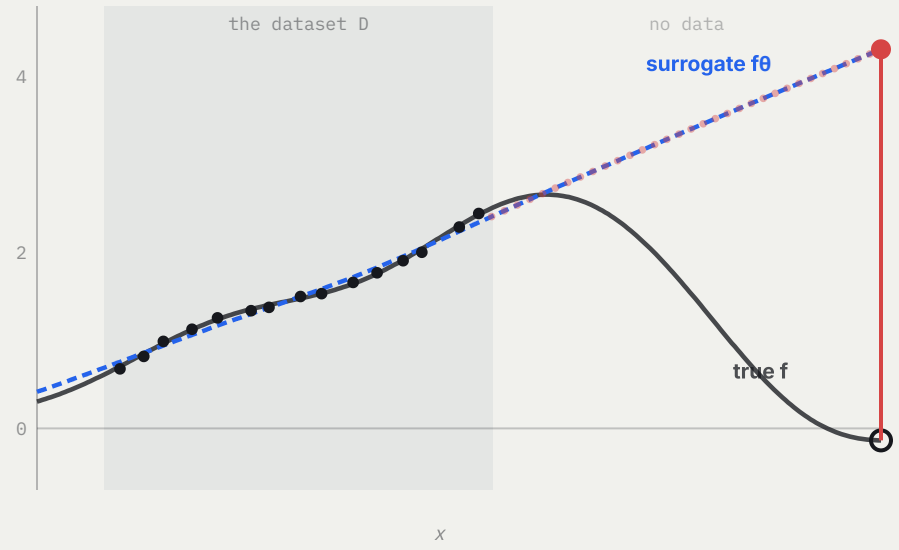
The optimiser is not a user of the surrogate. It is **an adversarial attack on it.**

Watch it happen

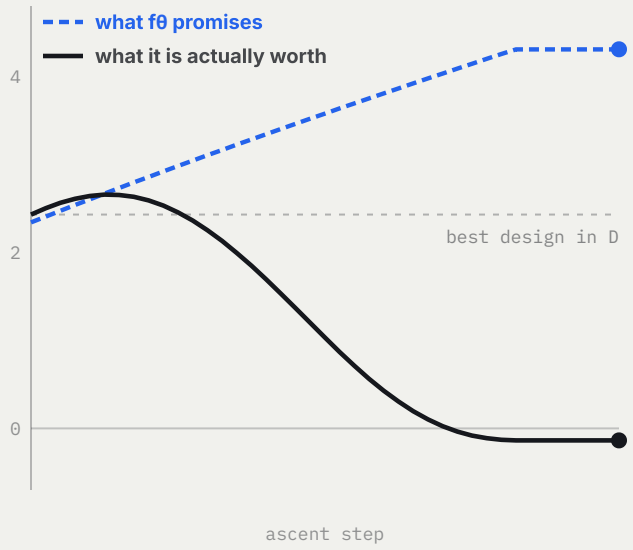
FIT ONCE, THEN ASCEND

STEP **40 / 40**

THE DESIGN SPACE



VALUE ALONG THE ASCENT



fit RMSE on D **0.058**

best design in D **2.43**

x_t = **10.00**

surrogate says **4.31**

truth is **-0.14**

over by 4.44

vs. the data: **-2.57**

Every further step raises the promise and lowers the payoff. The returned design is **worse than one we already owned**.

The surrogate fits the fifteen data points to an RMSE of 0.058 and then, off the data, keeps climbing. Ascent from the best design in D improves the true value for about five steps — and then spends the next thirty walking downhill in reality while the surrogate reports steady progress. **The returned design scores -0.14 where the surrogate promised 4.31** , and is worse than the design we already had.

The cure is not a better optimiser

Two repairs suggest themselves before the right one, and both fail for the same reason: they treat the search as the problem.

SEARCH HARDER

A stronger, more thorough optimiser finds a *higher* point of f_θ — which on this surface means a point still further from the data, and still more badly overestimated.

Optimisation strength is on the **wrong side** of the problem.

SEARCH LESS

Constrain the search to stay near D and the answer is capped at the best design already in the dataset.

Safe and pointless: **beating D was the entire task.**

Neither the optimiser nor its leash. The cure is a **more honest surrogate** — one whose maximum sits where the evidence can support it.

ACT 3

Conservative objective models

Q3. If the optimiser will attack the surrogate, train the surrogate against that attack.

COMs — train the surrogate to distrust its own optimiser

Conservative Objective Models · Trabucco, Kumar, Geng & Levine, ICML 2021

Q1 What changes offline? → Q2 Why does it fail? → **Q3 How do we fix it?** → Q4 What else helps?

We want a model that **does not overestimate the very inputs an optimiser would chase**. The obstacle is knowing which inputs those are — and the answer is to generate them, by simulating the attack we fear.

$$\mu(x) = \left\{ \sum_{t \geq 0} \delta_{x_t} \;:\; x_{t+1} = x_t + \eta \nabla_{x_t} f_\theta(x_t), \quad x_0 \sim D \right\}$$

Start from the data
$x_0 \sim D$ — real designs

→

Run the attacker
a few steps of ascent on the *current* surrogate

→

Collect what it visited
that set is $\mu(x)$: the inputs this surrogate is tempting

Because f_θ changes at every training step, μ is regenerated as training proceeds — an inner adversary chasing an outer defender, exactly as in adversarial training for robustness.

The loss, term by term

$$L(\theta) = \underbrace{\tfrac{1}{2}\,\mathbb{E}_{(x,y)\sim D}\left[\left(f_\theta(x) - y\right)^2\right]}_{\text{(i) fit the data}} + \alpha\Big(\underbrace{\mathbb{E}_{x\sim\mu(x)}[f_\theta(x)]}_{\text{(ii) push the adversaries down}} - \underbrace{\mathbb{E}_{x\sim D}[f_\theta(x)]}_{\text{(iii) hold the data up}} \Big)$$

- **(i)** ordinary regression — be right about the designs we actually measured;
- **(ii)** the conservative term — *prevents overestimation of out-of-distribution inputs*;
- **(iii)** the counter-term — without it, (ii) would drag the whole surface down and the model would simply predict $-\infty$ everywhere. It *prevents underestimation of in-distribution inputs*.

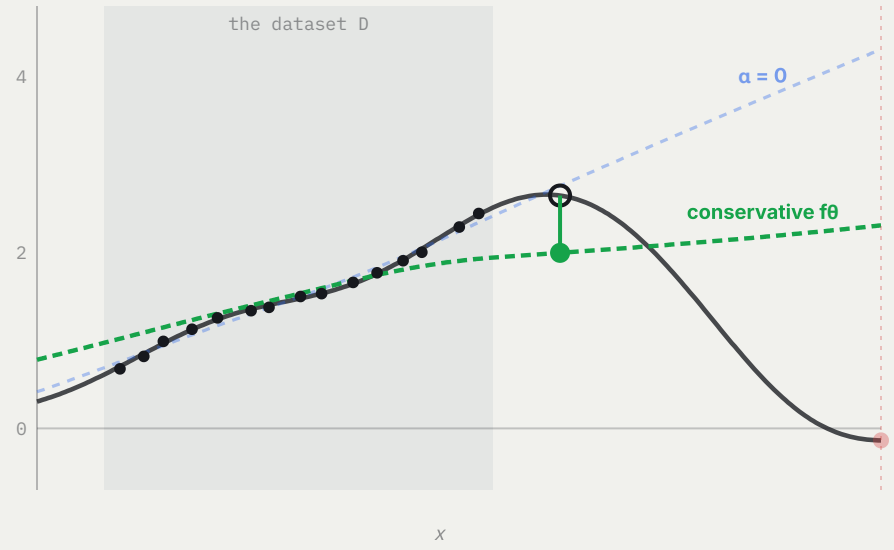
Structurally this is ordinary supervised regression plus one adversarial term. No new optimiser, no inversion, no sampler — which is exactly why COMs is the easiest of the three methods in this lecture to deploy. Optimising it is a naive gradient ascent started from **the best design already in D**.

Turning the dial

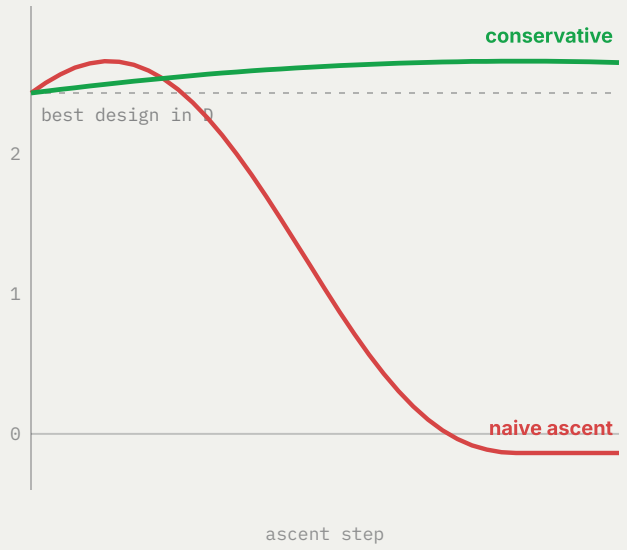
CONSERVATISM, DIALLED

CONSERVATISM **α = 0.30**

THE SURROGATE, RETRAINED



TRUE VALUE ALONG THE ASCENT



x* = **6.20**

surrogate says **1.99**

truth is **2.65**

under by 0.65

vs. the data: **+0.22**

The surrogate now **under**-promises at x* — a lower bound, so what it reports can be trusted.

The same dataset, the same optimiser, the same fifteen points — only the training loss differs. At $\alpha = 0$ the search runs to the boundary and returns a design worth -0.14. By $\alpha = 0.3$ it halts at 6.20, all but exactly the true optimum at 6.04. Watch the readout: past $\alpha \approx 0.15$ the surrogate's prediction at x^* falls *below* the truth. **It has become a lower bound** — and then, at $\alpha = 1.3$, so conservative that it will not leave the data at all.

Why it works — a learned lower bound

PROPOSITION 1 *(INFORMAL)* Trabucco et al., 2021

Under regularity assumptions, if α is large enough then the converged conservative model, evaluated at the designs its own optimiser produces, satisfies

$$\mathbb{E}_{x_0 \sim D,\, x_T \sim \mu(x_T|x_0)}\left[f_\theta(x_T)\right] \;\leq\; \mathbb{E}_{x_0 \sim D,\, x_T \sim \mu(x_T|x_0)}\left[f(x_T)\right]$$

Read the direction carefully. It does not say the surrogate is accurate. It says that **wherever the optimiser can reach, the surrogate under-promises** — so a design that looks good on the surrogate cannot be a hallucination, only an underestimate. The hallucinated peaks have been flattened, and gradient ascent has nowhere false left to climb.

Conservatism is a dial, and both ends are bad

choosing α is hard — so make it a constraint instead

$$\theta^* = \arg\min_\theta \tfrac{1}{2}\mathbb{E}_{(x,y)\sim D}\left[\left(f_\theta(x) - y\right)^2\right] \quad \text{s.t.} \quad \mathbb{E}_{x\sim\mu(x)}[f_\theta(x)] - \mathbb{E}_{x\sim D}[f_\theta(x)] \;\leq\; \tau$$

α is a penalty weight whose right value depends on the scale of y; τ is a *budget* on how far the surrogate may over-rate an adversary, and it is read in the units of the objective. Changing τ does not corrupt the loss value, so runs remain comparable.

T TOO LARGE

The constraint never binds. We are back to the naive fit, and the optimiser escapes.

T TOO SMALL

The surrogate flattens so hard that ascent cannot move. In the source ablation, $\tau = 0.1$ leaves the Hopper return pinned near its starting value for all fifty steps.

Converting a penalised objective into a constrained one to get a scale-free, tunable hyperparameter is a move this course has made before and will make again — **it is the trust region** of Lecture 1, and it is how TRPO will tame the policy gradient in Lecture 10.

The same disease, one rung up the course

this is the rhyme Lecture 12 will name

COMS — A CONSERVATIVE *OBJECTIVE*

$$\theta^* = \arg\min_{\theta} \tfrac{1}{2}\mathbb{E}_D\left[(f_\theta(x) - y)^2\right] + \alpha\left(\mathbb{E}_{\mu(x)}[f_\theta] - \mathbb{E}_D[f_\theta]\right)$$

The optimiser exploits f_θ at **out-of-distribution inputs**.

CQL — A CONSERVATIVE *VALUE*

$$Q^* = \arg\min_{Q} \tfrac{1}{2}\mathbb{E}_D\left[(Q - \mathcal{B}^\pi\hat{Q})^2\right] + \alpha\,\mathbb{E}_{s\sim D}\left(\log\textstyle\sum_a e^{Q(s,a)} - \mathbb{E}_{a\sim\hat{\pi}_\beta}[Q(s,a)]\right)$$

The policy exploits Q at **out-of-distribution actions**.

Fit the data · push down what the optimiser would chase · hold up what the data actually contains. **One shape, twice.**

Lecture 12 meets this failure again with a policy in place of an optimiser and a Q-function in place of a surrogate, and answers it with the identical three terms. When it does, it will quote this slide.

ACT 4

Other ways to be robust

Q4. Conservatism is one answer to overestimation. Two others attack the same disease from different sides.

Three cures for one disease

Q1 What changes offline? → Q2 Why does it fail? → Q3 How do we fix it? → **Q4 What else helps?**

METHOD	WHAT IT TREATS OVERESTIMATION AS	THE LEVER
NEMO *(Fu & Levine, ICLR 2021)*	a failure of **uncertainty** — the model does not know what it does not know	a normalised-maximum-likelihood posterior
COMs *(Trabucco et al., ICML 2021)*	a failure of **calibration on the attack** — the model over-rates what the optimiser finds	an adversarial penalty, α
RoMA *(Yu, Ahn, Song & Shin, NeurIPS 2021)*	a failure of **smoothness** — spurious spikes between and beyond the data	a local smoothness prior at the current candidate

They are not rivals so much as three readings of the same sentence: **the surrogate is unconstrained where there is no evidence**, and something must constrain it.

NEMO — how easily could the model have been talked into it?

The conditional NML distribution is the estimator closest to maximum likelihood **when the test label is chosen adversarially**:

$$p_{\mathrm{NML}}(y \mid x) = \frac{p\big(y \mid x;\ \hat{\theta}_{D\cup(x,y)}\big)}{\displaystyle\int p\big(y' \mid x;\ \hat{\theta}_{D\cup(x,y')}\big)\, dy'}$$

Pick a candidate label y'
for the query point x

→

Refit
$\hat{\theta}_{D\cup(x,y')}$ — the MLE on the data *plus that made-up point*

→

Ask how well it fits
$p(y' \mid x; \hat{\theta}_{D\cup(x,y')})$

→

Normalise over all y'
the answer is the posterior

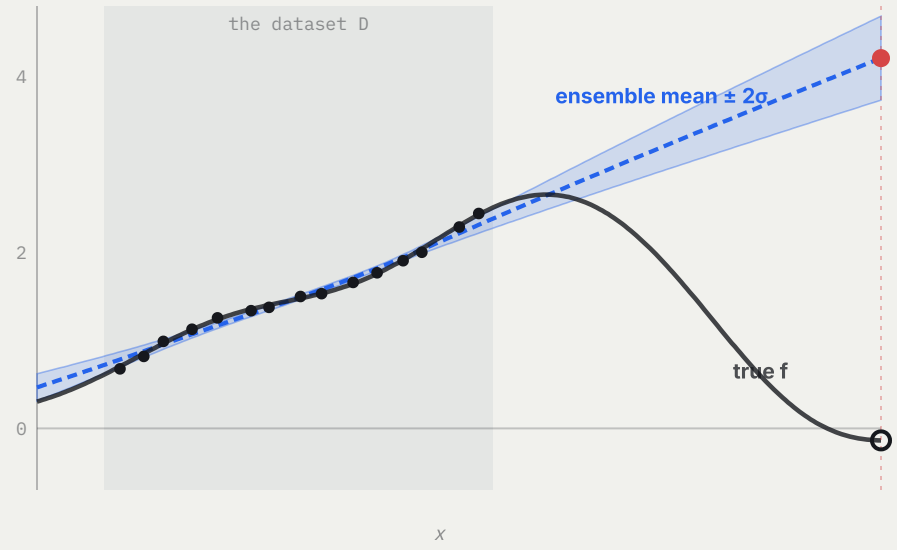
Far from the data, *every* candidate label can be accommodated almost perfectly — one extra point barely moves a flexible model — so the normalised distribution comes out wide. Near the data, only labels close to the trend survive the refit, and it comes out narrow. **Uncertainty is measured as how easily the model could have been talked into any answer.** The integral is intractable, so NEMO quantises y into K bins, keeps K models, and updates them incrementally *while* it optimises x rather than rebuilding them at each iterate.

But surely an ensemble would have caught it?

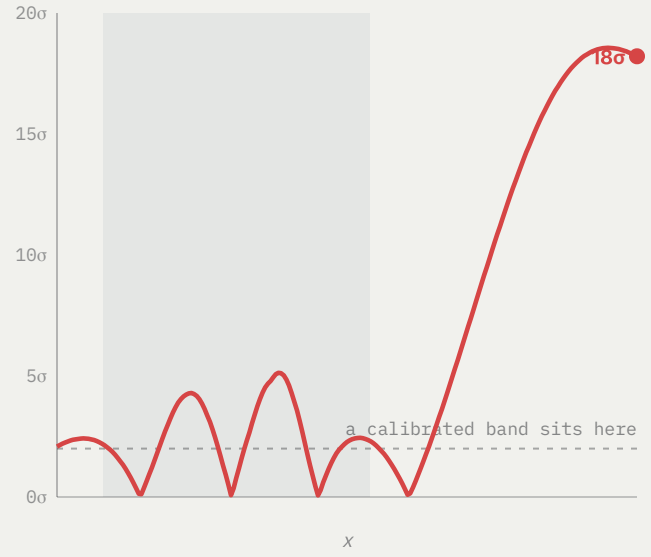
WOULD AN ENSEMBLE HAVE CAUGHT IT?

MEMBERS 10

WHAT THE MEMBERS AGREE ON



THE TRUTH, IN THEIR OWN SD



inside D

spread **0.024** · error **0.053**

outside D

spread **0.149** · error **1.93**

spread ×**6.2**, error **×37**

Ascent on the ensemble mean returns x* = **10.00**, where the truth lies **18σ** outside the band. Try more members.

Ten surrogates, each fitted to a bootstrap resample of the same fifteen points. Out of distribution their spread does widen — by about six times. Their actual error grows **thirty-seven times**. At the design their own averaged optimiser returns, the truth sits eighteen standard deviations outside the band they agree on. The alarm fires; it is simply far too quiet, because the members share an architecture and so extrapolate wrongly *together*.

RoMA — flatten the surface the optimiser is standing on

The clause NEMO and COMs leave implicit: a deep network overestimates off-distribution **because it is not smooth**. A flexible model threaded through sparse data does not interpolate gently; it oscillates, and its spurious spikes are precisely what an argmax finds.

STAGE 1 — TRAIN IT SMOOTH

$$L(\theta) = \max_{\tilde{\theta} \in B(\theta)} \mathbb{E}_{(x,y)\sim D,\, \delta \sim \mathcal{N}(0,\sigma)} \Big[\big(f(x+\delta; \tilde{\theta}) - y \big)^2 \Big]$$

Gaussian smoothing of the *inputs* under worst-case *weight* perturbations, $B(\theta) = \{\tilde{\theta} : \|\theta_l - \tilde{\theta}_l\|_F \leq \epsilon \|\theta_l\|_F\}$; the inner maximisation by projected gradient descent.

STAGE 2 — RE-SMOOTH AS YOU GO

$$\theta_t = \arg\min_{\tilde{\theta} \in B(\theta)} \big\| \nabla_x f(x; \tilde{\theta}) \big\|_2 \Big|_{x = x^{(t)}} + \alpha \big(f(x^{(t)}; \tilde{\theta}) - f(x^{(t)}; \theta_{t-1}) \big)^2$$

Stage 1 only smooths where the data is. So at *every* ascent step, re-adapt the model to be flat at the current candidate — first term for smoothness, second to stop θ drifting.

The source figure says it in two panels: without the prior, a jagged surrogate's tallest spike is a *wrong solution*; with it, the surrogate lies on the truth and the argmax is the **right one**.

What the benchmark says

100th-percentile score on Design-Bench, normalised so the best design in the dataset = 1.000

METHOD	GFP	MOLECULE	SUPERCOND.	HOPPER	ANT	DKITTY	AVG
Dataset max	3.152	6.558	73.90	1361.6	108.5	215.9	*1.000*
Gradient ascent	2.894	6.636	89.64	1050.8	399.9	390.7	1.237
MINs	3.315	6.508	80.23	746.1	388.5	352.9	1.304
CbAS	**3.408**	6.301	72.17	547.1	393.0	**396.1**	1.324
COMs	3.305	6.876	110.0	**2395.7**	378.8	341.4	1.589
NEMO	3.359	6.682	**127.0**	2130.1	393.7	**431.6**	1.687
RoMA	3.357	**6.890**	103.9	**2466.5**	**468.5**	384.3	**1.705**

Naive gradient ascent is not useless — it is the *worst* of the six, and on HopperController it returns less than the best trajectory already in the dataset. Every method that beats it does so by **adding a constraint on what the surrogate is allowed to believe**, not by searching harder.

All three say the same thing

Respect uncertainty off-distribution, or the optimiser will weaponise it.

BAYESIAN OPTIMISATION *(LEC 4)*

Uncertainty is an **opportunity**: go where the band is wide, because a query there teaches you the most. A wrong belief costs one round.

OFFLINE MBO *(LEC 5)*

Uncertainty is a **hazard**: stay away from where the band is wide, because nothing will contradict you there. A wrong belief is the answer you ship.

Same Gaussian-process-era intuition, opposite operational consequence — which is why the offline methods are built around *conservatism* where BO was built around *exploration*. Lecture 4's lesson, made non-negotiable.

CLOSING

Closing

A forward model, searched. Lecture 6 inverts every word of that.

It is already in production

PRIME — HARDWARE ACCELERATORS *(ICLR 2022)*

A conservative surrogate in the COMs shape, trained over *contexts* (target workloads) and given infeasible layouts as extra negatives:

$$\theta^* = \arg\min_{\theta} \mathcal{L}(\theta) - \beta\, \mathbb{E}_{x' \sim D_{\text{infeasible}}}[f_\theta(x')]$$

On a U-Net + t-RNN target, latency ≈ 745 against a simulator-driven baseline's ≈ 1080 — with the simulator never called.

LCOMS — CRYSTAL STRUCTURES *(ICLR 2022)*

Chemical space is not a vector space, so a **CD-VAE encoder** $\phi(x, c)$ maps a crystal into one, and the conservative surrogate of the lattice energy is optimised **inside that latent space**, then decoded.

Mean energy improvement 2.25 against supervised learning's 1.10.

Read the second one twice. A generative model supplies the coordinates; a conservative surrogate does the searching. Forward-and-search and inverse-and-sample are not rivals — **the second can be the first's coordinate system**, which is one reason Lecture 6 follows immediately.

Where we are — a forward model, then a search

	THE MODEL	THE DECISION
Surrogate *(Lec 5 ✓)*	forward: $f_\theta(x) \approx f(x)$	optimise / search over f_θ
Generative *(Lec 6)*	inverse: $p(x \mid y)$	sample a good design

What this lecture hands on is **a forward model then a search, and the warning that the optimiser is an adversary**: build a *forward* surrogate of f, then *search* it — carefully, conservatively, so that the search cannot exploit our ignorance.

But there is a completely different route. Instead of approximating f and searching it, why not learn to **generate** good designs directly — the inverse map from "I want a high value" to "here is an input that gives it"? That is Lecture 6: don't search the design, **produce** it. Hold the shape of this lecture, because the duality returns in Part IV as value-based RL (search a value) against policy-based RL (produce an action).

Offline, a surrogate's optimism becomes the optimiser's trap.

The cure is conservatism: teach the model to doubt itself exactly where it will be attacked, and gradient ascent has nowhere false left to climb.

Questions?

Two things to carry out of here. **The optimiser is an adversary** — you will meet it again in Lecture 12 as a policy exploiting a Q-function. And **a forward model plus a search** — you will meet its inverse in Lecture 6, next.

APPENDIX

Backup slides

Complete statements, kept out of the narrative.

Backup 1 — offline MBO against Bayesian optimisation

The same goal, $\arg\max_x f(x)$, under different access — and the difference dictates the method.

	BAYESIAN OPTIMISATION *(LEC 4)*	OFFLINE MBO *(LEC 5)*
data	*active* — query f each round	*fixed* — a static dataset D
a mistake is	corrected by the next query	**uncorrectable** — it is returned as the answer
uncertainty drives	*where to sample*	*where not to trust*
the core risk is	slow convergence	overestimation → an invalid design
the design principle	exploration	**conservatism**

Both build a surrogate; both maximise something over it. What differs is whether the loop closes. When it does, optimism is self-correcting and therefore cheap — an over-rated point gets queried, found wanting, and the posterior repairs itself. When it does not, optimism is a one-way door.

Backup 2 — generating the adversarial distribution

$$\mu(x) = \left\{ \sum_t \delta_{x_t} \;:\; x_0 \sim D, \quad x_{t+1} = x_t + \eta \nabla_{x_t} f_\theta(x_t) \right\}$$

Reading it. Start from real data points; run a few steps of gradient ascent on the *current* surrogate; collect what it visits. These are exactly the inputs this surrogate would lure an optimiser toward, so these are the inputs whose predicted value must come down.

ALGORITHM 1 — TRAINING

```
initialise f_θ; pick η, α
for i = 1 … steps:
  sample (x₀, y) ~ D
  x_T ← ascent from x₀ on f_θ
  μ ← Σ_{x₀∈D} δ_{x_T(x₀)}
  L = E_D (f_θ(x₀)−y)²
      − α E_D[f_θ] + α E_μ[f_θ]
  θ ← θ − λ ∇_θ L
```

ALGORITHM 2 — FINDING X^*

```
x̃ = argmax_{(x,y)∈D} y   ← the best
                            design we own
for t = 0 … T−1:
  x_{t+1} = x_t + η ∇_x f_θ*(x_t)

return x* = x_T
```

Note where Algorithm 2 starts. Ascent is initialised at the **best design in the dataset**, not at random — so the method's worst case is roughly "return what we already had", and the conservative penalty is what stops it wandering away from that floor.

Backup 3 — the COMs loss, term by term, and the guarantee

$$L(\theta) = \underbrace{\tfrac{1}{2}\mathbb{E}_{(x,y)\sim D}\left[(f_\theta(x)-y)^2\right]}_{\text{(i) supervised fit}} + \alpha \underbrace{\mathbb{E}_{x\sim\mu(x)}[f_\theta(x)]}_{\text{(ii) push adversaries down}} - \alpha \underbrace{\mathbb{E}_{x\sim D}[f_\theta(x)]}_{\text{(iii) hold data up}}$$

- **(i)** ordinary regression: be accurate on the data we have.
- **(ii)** the conservative term: minimise the predicted value on $\mu(x)$, the points gradient ascent on f_θ actually produces. *Prevents overestimation of out-of-distribution inputs.*
- **(iii)** the counter-term: without it, (ii) drags the entire surface down, data included. Maximising the predicted value on D *prevents underestimation of in-distribution inputs*.

The guarantee. Under regularity assumptions, the conservative iterate satisfies, for all $x \in D$ and $x'' \in \mathcal{X}$,

$$f_\theta^{k+1}(x'') := \max\left\{ f_\theta^{k+1}(x) - \hat{L}\|x''-x\|_2,\ \ \tilde{f}_\theta^{k+1}(x'') - \eta\alpha\,\mathbb{E}_{x\sim\bar{D},x'\sim\mu}\left[G_f^k(x'',x')\right] + \eta\alpha\,\mathbb{E}_{x\sim\bar{D},x'\sim\bar{D}}\left[G_f^k(x'',x')\right]\right\}$$

where $\tilde{f}_\theta^{k+1}$ is the iterate that *would* have resulted without conservative training. For α large enough the asymptotic model therefore lower-bounds the truth on whatever the optimiser reaches: $\mathbb{E}[f_\theta(x_T)] \le \mathbb{E}[f(x_T)]$.

Backup 4 — NEMO, made tractable

The estimator. p_{NML} is the minimax solution of $\arg\min_h \max_{y'} \left(\log p(y' \mid x; \hat{\theta}_{D\cup(x,y')}) - \log h(y' \mid x)\right)$, where $\hat{\theta}_{D\cup(x,y)} = \arg\max_\theta \frac{1}{N+1} \sum_{(x,y)\in D\cup(x,y)} \log p(y \mid x, \theta)$.

Problem. The denominator requires training an MLE *for every possible* y and then integrating over them — impossible twice over for a deep network.

Fix 1 — quantise. Floor each y into one of K bins, so the integral becomes a sum: $\int_y p(y \mid x, \hat{\theta}_{D\cup(x,y)})\, dy \approx B \sum_{k=1}^{K} p\big(\lfloor y_k \rfloor \mid x, \hat{\theta}_{D\cup(x,\lfloor y_k \rfloor)}\big)$.

Fix 2 — amortise. Keep K models and update them incrementally *while* optimising x, rather than retraining from scratch at each iterate:

```
for t = 1 … T:
  for k = 1 … K:  D' ← D ∪ (x_t, ⌊y_k⌋);  θ^k ← θ^k + α_θ ∇ LogLik(θ^k, D')
  p̂_NML(y | x_t) ∝ p(y | x_t, θ^y) / Σ_k p(⌊y_k⌋ | x_t, θ^k)
  x_{t+1} ← x_t + α_x ∇_x E_{y ~ p̂_NML(y|x)}[ g(y) ]
```

Quantisation flattens the landscape and kills the gradient, so NEMO's head outputs one minus the CDF of a *logistic* distribution sampled at intervals of $1/K$ and takes the mean; gradients then flow through the logistic mean $\mu(x)$, and Proposition 4.1 guarantees $\langle \nabla_x \mu(x), \nabla_x y_{\mathrm{mean}}(x) \rangle \geq 0$ — the smooth surrogate gradient never points against the one we want.